```
import numpy as np
import matplotlib.pyplot as plt

def simulate_rel_clock_interferometer(
    p1=0.5,
    p2=2.0,
    mass=1.0,
    c=1.0,
    omega0=10.0,
    Delta_n=5.0,
    t_max=20.0,
    num_steps=500
):
    """
    Simulates a relativistic clock interferometer where a particle carrying an
    internal quantum clock (Hardy space H^2(S^1) with N levels and energy spread Delta_n)
    is placed in a momentum superposition c1|p1> + c2|p2>.

    Parameters:
    - p1, p2: Momentum states of the two interferometer arms
    - mass: Particle rest mass m
    - c: Speed of light
    - omega0: Internal proper-clock frequency omega_0
    - Delta_n: Clock number state spread (determines internal coherence width)
    - t_max: Maximum coordinate evolution time
    - num_steps: Time resolution points
    """
    # Time array
    t = np.linspace(0.0, t_max, num_steps)

    # 1. Compute relativistic Lorentz factors for each arm
    # gamma(p) = sqrt(1 + p^2 / (m^2 c^2))
    gamma1 = np.sqrt(1.0 + (p1**2) / (mass**2 * c**2))
    gamma2 = np.sqrt(1.0 + (p2**2) / (mass**2 * c**2))

    # Proper-time dilation difference per unit coordinate time: delta_inv_gamma = gamma1^-1 - gamma2^-1
    delta_inv_gamma = (1.0 / gamma1) - (1.0 / gamma2)

    # 2. Internal clock population envelope |c_n|^2 (Gaussian in number basis around n0)
    n_max = 50
    n = np.arange(n_max)
    n0 = n_max // 2
```

```
    # Population weights |c_n|^2 normalized to sum to 1
    c_sq = np.exp(-((n - n0)**2) / (2.0 * Delta_n**2))
    c_sq /= np.sum(c_sq)

    # Equal superposition weights in spatial arms: c1 = c2 = 1/sqrt(2)
    c1_amp = 1.0 / np.sqrt(2.0)
    c2_amp = 1.0 / np.sqrt(2.0)

    # 3. Compute exact interferometric visibility magnitude over time
t
    # Eq. (50) prediction: V(t) = | sum_n |c_n|^2 ( |c1|^2 e^{-i n
omega_0 gamma1^{-1} t} + |c2|^2 e^{-i n omega_0 gamma2^{-1} t} ) |
    # Which simplifies for equal arms to: V(t) = | sum_n |c_n|^2 cos(n
omega_0 delta_inv_gamma t / 2) | (ignoring global phase)
    # More generally, compute the off-diagonal coherence matrix
element magnitude |rho_int^{n,n}(t)| averaged over n:

    visibility = np.zeros_like(t)
    for idx, time_val in enumerate(t):
        # Sum over clock number states n weighted by population
|c_n|^2
        # Phase difference accumulated between arm 1 and arm 2 for
clock state |n> is n * omega0 * delta_inv_gamma * t
        phase_diff = n * omega0 * delta_inv_gamma * time_val

        # Interference sum: V(t) = | sum_n |c_n|^2 exp(-i n omega0
delta_inv_gamma t) |
        complex_sum = np.sum(c_sq * np.exp(-1j * phase_diff))
        visibility[idx] = np.abs(complex_sum)

    # Analytical Gaussian envelope approximation: V_approx(t) ~ exp(-
(Delta_n * omega0 * delta_inv_gamma * t)^2 / 2 )
    envelope_approx = np.exp(-0.5 * (Delta_n * omega0 *
delta_inv_gamma * t)**2)

    return t, visibility, envelope_approx, gamma1, gamma2,
delta_inv_gamma

if __name__ == "__main__":
    # Run simulation with default physical parameters
    t, vis, env, g1, g2, d_inv_g = simulate_rel_clock_interferometer(
        p1=0.2, p2=2.5, mass=1.0, c=1.0, omega0=8.0, Delta_n=6.0,
t_max=15.0
    )

    print(f"Arm 1 Lorentz factor gamma1 = {g1:.4f} (1/gamma1 =
{1/g1:.4f})")
    print(f"Arm 2 Lorentz factor gamma2 = {g2:.4f} (1/gamma2 =
```

```
{1/g2:.4f})")
    print(f"Proper-time differential rate delta_inv_gamma =
{d_inv_g:.4f}")

    # Plotting results
    plt.figure(figsize=(9, 5))
    plt.plot(t, vis, label=r"Quantum Interferometric Visibility $V_{int}(t)$",
color="#1f77b4", lw=2)
    plt.plot(t, env, label=r"Gaussian Decay Envelope
$\exp\left(-\frac{1}{2}\Delta n^2\omega_0^2(\gamma_1^{-1}-\gamma_2^{-1})^2t^2\right)$", color="#d62728", ls="--", lw=1.8)
    plt.axhline(0, color="gray", ls=":", alpha=0.5)

    plt.xlabel(r"Coordinate Time $t$", fontsize=11)
    plt.ylabel(r"Clock Coherence / Visibility $V$", fontsize=11)
    plt.title(r"Relativistic Clock Interferometer Visibility Drop (
$p_1 = 0.2, p_2 = 2.5, \Delta n = 6$)", fontsize=12)
    plt.ylim(-0.05, 1.05)
    plt.legend(loc="upper right", fontsize=10)
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()
```